ConfigStream

Architecture for trustworthy temporal history

Kafka, stateful Spark, committed Parquet and explainable queries

Group 4 | Big Data Storage & Processing
Phạm Công Hoàng Lê Tiến Họp Mai Vũ Hoàng Nguyễn Đức Dũng

FIRST MILESTONE / 26 OCTOBER 2026

Configuration change: before and after

Router R-001 / version 10

```
hostname router-001
logging enabled
telnet disabled
```

20 Oct 2026 / 08:00 UTC

Router R-001 / version 11

```
hostname router-001
logging disabled
telnet enabled
```

20 Oct 2026 / 09:00 UTC

What changed? When? On which asset?

A latest-state file cannot answer these questions without versioned evidence and comparison logic.

SOURCE / SCOPE: Illustrative example adapted from processing/demo.py; timestamps are explanatory, not source measurements.

The product: three linked capabilities

01

Preserve history

Snapshots with identity, version, time and provenance

02

Explain changes

Compare adjacent versions and evaluate explicit rules

03

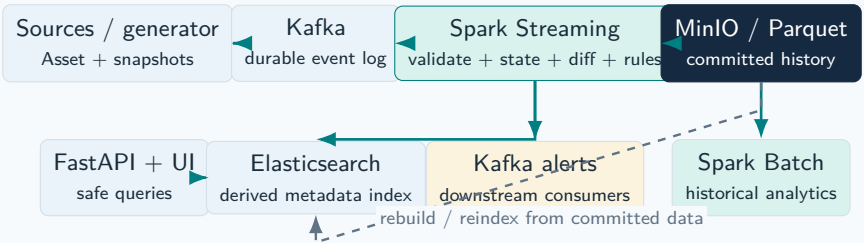
Search & analyze

Query safe metadata and study temporal patterns

Core design question How do we preserve correct temporal meaning under retries, late arrivals and failures?

SOURCE / SCOPE: schemas/; processing/; spark/streaming/; api/routes/documents.py.

End-to-end architecture: configuration path



Separation of responsibilities: Kafka transports events; Spark derives changes; MinIO keeps authoritative history; Elasticsearch/FastAPI serve safe metadata; batch jobs read committed Parquet.

SOURCE / SCOPE: docs/architecture.md; spark/streaming/outputs.py; docs/api.md; docs/data-lake.md.

Boundary 1 / Kafka: decouple producers from processors

SOURCE MESSAGES

Three input topics

`config.assets` identity / metadata

`config.events` audit of changes

`config.snapshots` versioned content

Partition key: `asset_id`

Consistent keying groups records by asset.

TOPIC PARTITIONS / OFFSETS

P0	v1	v3	v2
P1	r1	r2	r3
P2	s1	s2	s3

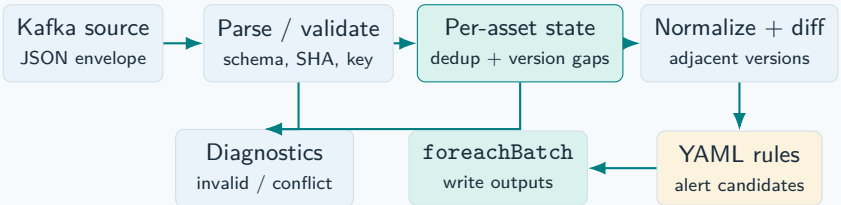
Offset: position within a partition.

Spark reads `config.snapshots` and checkpoints its progress.

Important: Kafka partition order does not imply source-version order; keying by asset does not solve delayed/replayed source versions.

SOURCE / SCOPE: messaging/kafka_producer.py; docs/kafka-topics.md; spark/streaming/kafka_source.py.

Boundary 2 / Spark: from bytes to meaningful changes

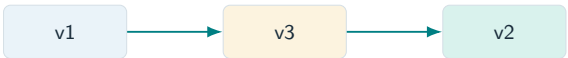


Why stateful? Compare only $v_i \rightarrow v_{i+1}$ for the same asset, despite retries, duplicate IDs and late arrival order. Event-time watermark + eviction bound memory use.

SOURCE / SCOPE: spark/streaming/main.py; validation.py; stateful.py; processing/state,normalization,diff,rules.py.

Late arrivals: restore version order

Observed arrival order — Kafka messages may carry out-of-order source versions



Stateful version matching — Per-asset bounded buffer



Rule: emit (v_i, v_{i+1}) only if both endpoints exist and that logical edge has not already been emitted.

Retention and watermark policies bound late repair; they do not make late-data recovery unlimited.

SOURCE / SCOPE: processing/state.py; spark/streaming/stateful.py; scripts/kafka_stateful_demo.py.

Diffs become explainable alerts

Version 10 → 11

```
- logging enabled
+ logging disabled

- telnet disabled
+ telnet enabled
```

Normalization removes selected textual noise and masks supported secret patterns before diffing.

RULE MATCH / HIGH

Logging disabled
Regex on added diff lines

RULE MATCH / CRITICAL

Telnet enabled
Regex on added diff lines

Interpretation boundary: a rule alert is not proof of an incident; line diff is not a vendor-semantic configuration parser.

SOURCE / SCOPE: processing/demo.py; processing/diff.py; rules/default.yaml; processing/normalization.py.

Boundary 3 / MinIO: how the Parquet history is laid out

ONE S3-COMPATIBLE BUCKET

```
config-lake/
raw/snapshots/
processed/snapshots/
processed/diffs/
processed/alerts/
processed/diagnostics/
analytics/...
```

Each micro-batch is scoped by query_id/batch_id; data files are partitioned by year/month/day.

RAW

Keep Kafka payload bytes with topic, partition and offset provenance.

PROCESSED

Typed normalized snapshots, diffs, alerts and diagnostics stored as Snappy Parquet.

Three different partitions: Kafka partition (event log), Spark execution partition (parallel tasks), and date partition (object paths).

SOURCE / SCOPE: spark/streaming/sinks/lake.py; docs/data-lake.md; docs/pipeline-integrity-amendment.md.

Boundary 3 / Commit: make complete batches visible



What readers can trust

Manifest-declared, verified files form the accepted batch. Matching retry can be recognized; a partial write is not published as committed.

What still remains open

No atomic transaction across MinIO, Elasticsearch and Kafka. Kafka alerts can be physically duplicated on some retries.

SOURCE / SCOPE: spark/streaming/sinks/lake.py; spark/streaming/outputs.py; docs/pipeline-integrity-amendment.md.

Boundary 4 / Serving: one source of truth, two readers



Data boundary: API is read-only and does not return raw configuration bodies. Dashboard cross-domain research cards come from accepted evidence bundles, not a unified live stream for all domains.

SOURCE / SCOPE: docs/api.md; docs/historical-analytics.md; docs/elasticsearch.md; frontend/public/research/v1/catalog.json.

From configuration history to four temporal contracts

TERRAFORM / FILE VERSIONS

Adjacent version diff

Compare v_i with v_{i+1}, not Git commits with deployments.

RPKI / DAILY STATE

Set membership transitions

Differences between complete observed days, not operator actions.

RIS / BGP OCCURRENCES

Preserve event multiplicity

A repeated announcement may be a separate real occurrence.

DNS / MEASUREMENT

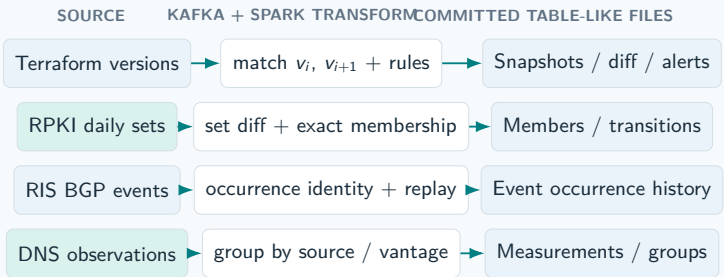
Observed query responses

Vantage/coverage limits prohibit global-state claims.

What is shared: Kafka/Spark execution, typed records, provenance, committed Parquet. **What differs:** identity, timing, dedup, and what a "change" means.

SOURCE / SCOPE: frontend/public/research/v1/catalog.json; docs/phase20-cross-domain-analysis.md.

Four processing lanes: shared platform, separate logic



Not one universal diff algorithm: configuration rules apply to the versioned file lane. RIS, RPKI and DNS use separate domain definitions and proof obligations.

SOURCE / SCOPE: spark/streaming/; spark/rpki_stream.py; spark/ris.py; spark/dns.py; docs/architecture.md.

Prepared data: accepted measurements, not mixed units

10,188
Terraform accepted Git versions / 375 repos

14
Complete RPKI daily observations / 1–14 Sep

4,725,156
RIS event occurrences / 1 hour / rrc00

2,234,863
Raw DNS measurements / two source objects

Never sum these rows: file revisions, full daily state sets, event occurrences and measurements are different units. Controlled synthetic histories test retries and late delivery.

SOURCE / SCOPE: frontend/public/research/v1/catalog.json; accepted scope fixed per source and time window.

What is implemented: executable and integrated evidence

RUNNABLE PROCESSING DEMONSTRATION

```
make setup
make processing-demo
```

178

state, diff and rule unit tests passed, as recorded for 25 Oct 2026.

PREVIOUSLY RECORDED LOCAL REPLAY

10,188 snapshots

3,683 adjacent diffs

29.4 MB Parquet committed

Kafka → Spark → MinIO; 375 scoped Terraform repositories.

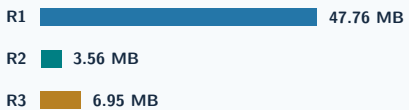
Do not conflate: the small Python demo needs no services; the integrated replay is a prior accepted local run, not a fresh live checkpoint test.

SOURCE / SCOPE: presentation README; docs/phase12-acceptance.md. 29,402,997 committed Parquet bytes.

One focused result: RPKI representation trade-off

SAME 14 COMPLETE DAILY OBSERVATIONS

R1 Full daily states
R2 Initial state + daily additions/removals
R3 R2 + extra full state on day 8



Warm reconstruction (R1/R2/R3): 6.73 / 6.88 / 6.76 s.

ANALYTICAL FILES ONLY

−92.6%

R2 vs R1 (47.76 → 3.56 MB)

ALL MEASURED LOCAL FILES

−17.5%

R2 vs R1 (251.98 → 207.90 MB)

SOURCE / SCOPE: workloads/results/phase20/rpki-representations.json and rpki-all-in-storage.json; excludes deployed broker/checkpoint bytes.

Architecture guarantees versus unproven claims

VALIDATED WITHIN ACCEPTED SCOPE

- Deterministic version adjacency / rules
- Committed Parquet and integrity checks
- Scoped historical replay and exact reads
- Separate temporal contracts for four sources

NOT YET ESTABLISHED

- Multi-node horizontal scaling or HA
- Cross-sink end-to-end exactly-once
- Production traffic realism or device impact
- Storage savings across every domain

Checkpoint conclusion: executable architecture with bounded correctness evidence; not a demonstrated production-scale configuration management service.

SOURCE / SCOPE: docs/phase12-acceptance.md; docs/phase20-acceptance.md; docs/phase20-k8s-acceptance.md.

Checkpoint readiness and next milestone

REQUIRED AT THIS CHECKPOINT

Source code
Runnable generator / processing demo, automated tests

Data readiness
Four scoped real-source corpora and manifests

System architecture
Kafka, four Spark paths, MinIO and serving boundary

NEXT MILESTONE: EVALUATION

01. Integrated reproducible demo
Source → Kafka → committed Parquet → API

02. Sustained workload tests
Logical throughput, latency, backlog, memory, storage

03. Failure / restart evidence
Replay and checkpoint behavior on latest revision

Questions and feedback welcome

SOURCE / SCOPE: https://github.com/kongwoang/ConfigStream; checkpoint scope: local / one-node accepted demonstrations.